What Makes a Video Game Successful?

Yi-Ju Chen, Yi-Jui Chen, Matthew Bellovich

Introduction

Tens of thousands of games are released on Steam, yet only a small share reach a large audience. Critic scores aggregated by Metacritic have traditionally served as the industry's signal of which games will succeed, but evidence on whether they actually track sales is mixed [1,2], and more importantly for Steam, most games never receive one. In our dataset, only about 2.6% of games released since 2015 have a Metacritic score. This project asks two questions. First, are critic scores and game characteristics associated with a game's commercial success? Second, what natural groupings of games exist on Steam, and do these groupings differ in commercial success?

Answering these questions matters most for developers, particularly small independent studios, who make pricing, localization, and platform decisions with limited budgets and little market data. Knowing which characteristics tend to accompany higher ownership, and which market segments exist, lets these teams benchmark their plans against comparable titles rather than relying on critic reception they may never receive.

We were motivated by this gap between how the industry measures success and what information is actually available for most games. As players, we have often come across games we loved that never found the audience they deserved, and wished their developers could have benefited from greater visibility. Understanding which characteristics are associated with commercial success could help such games reach more players.

In supervised learning, a random forest was the strongest of the three model families we compared. Game age and the number of supported languages were the most important predictors of ownership tier; whether a game had received a Metacritic score mattered more than the score itself. Overall predictive power remained modest. In unsupervised learning, both K-prototypes and latent class analysis identified structure in the catalog features, but the resulting groups showed only a weak association with estimated ownership. Together, these results suggest that the pre-release characteristics available in the current Steam dataset explain only a limited portion of commercial success. Additional signals available before release, such as wishlist counts, might improve prediction, yet commercial outcomes still depend on many factors beyond what can be observed in the catalog alone.

Related work

The Steam game critic score project is similar to our idea because it uses the same dataset and is attempting to answer the same business question. Instead of using Metacritic score to predict commercial success, it is trying to predict Metacritic score to find underrated games. Additionally, we use K-prototypes to group games based on a set of variables and examine whether such grouping can be a meaningful predictor of the estimated number of owners.

"Unveiling Success Drivers in Gaming: A Machine Learning Study Across Steam, Twitch, and Metacritic" uses machine learning models to identify factors associated with video game revenue. Our project focuses on predicting commercial success using estimated ownership rather than revenue, analyzing a much larger dataset of games released between 2015 and 2025. Unlike the article, we exclude post-release features that could introduce data leakage and examine how much predictive value Metacritic scores provide beyond other game characteristics.

"Video Game Sales Success Using Random Forest Based Machine Learning Techniques" uses multiple supervised learning models to classify games into high and low sales categories based on multiple game

characteristics. Our project expands on this research by using a larger, more recent dataset and incorporating a broader range of features. We also used more rigorous model evaluation techniques, including nested cross-validation, feature ablation, and sensitivity analysis, to better understand model performance and identify which characteristics contribute most to predicting commercial success.

Data Source

We used the Steam Games Dataset maintained by FronkonGames and hosted on Hugging Face. The dataset is released under the MIT license, loaded directly with the Hugging Face datasets library, and stored in Parquet format. Each row represents one Steam application, identified by its Steam app ID, and the dataset includes 41 columns covering games' description, pricing, platform support, review counts, playtime statistics. We restricted the data to approximately 119,000 games released between 2015 and 2025.

Feature Engineering

Our feature engineering process focused on transforming the raw Steam Games Dataset into a shared cleaned dataset, followed by additional processing specific to supervised and unsupervised learning. The pipeline included:

1. *Initial preprocessing*. Removed unnecessary columns and standardized column dtypes.
2. *Publisher & developer experience*. Engineer features describing prior game releases, using only information before each game's release date to avoid information leakage. These features were calculated before filtering the dataset by release year to preserve historical experience.
3. *Filter by release date*. Restricted the dataset to games released between 2015 and 2025.
4. *Generate other new features*. Created features describing game age, release timing, supported languages, genres, and Steam categories. Regional language variations were standardized, and binary indicators were created for selected languages, genres, and categories. Estimated ownership was condensed into three commercial success tiers: Low (<20K), Mid (20K–100K), and High (100K+).

The resulting dataset served as the starting point for additional feature engineering specific to each modeling approach. Complete inventories of the final model features are provided in Appendix A.1 and A.2 for supervised and unsupervised learning, respectively.

Supervised Learning

Workflow Overview

We used supervised learning to classify games into three commercial success tiers based on estimated ownership: Low (<20K), Mid (20K–100K), and High (100K+). Starting with 119,036 Steam applications released between 2015 and 2025, we removed 9,555 non-game items, leaving 109,481 games. Our workflow consisted of:

1. *Select features*. Chose game characteristics relevant to commercial success while excluding post-release metrics that could introduce data leakage.
2. *Split data*. Created a stratified 80% training and 20% test split, preserving class proportions and reserving the test set for final evaluation.
3. *Train and compare models*. Evaluated logistic regression, random forest, and K-nearest neighbors against a majority-class baseline using nested cross-validation, with preprocessing fitted separately within each training fold to prevent information from leaking between training and validation data.
4. *Tune and evaluate*. Selected random forest as the best-performing model, tuned it on the full training set, and evaluated it once on the held-out test set.

5. *Analyze performance*. Conducted feature importance, ablation, sensitivity analysis, tradeoffs, and failure analysis to understand model behavior and limitations.

Features

We chose features based on our research question and our understanding of the data rather than automated feature selection, then measured each feature's contribution afterward through ablation and permutation importance.

Metacritic score. Only 2.6% of games have a Metacritic score. The absence of a score is itself informative, since reviewed games tend to be more visible. We therefore represented it as two features. First, a binary flag for whether a game has a score. Second, the score itself. In the raw data, games without a score are recorded as 0, which a model would treat as an extremely low score, so we replaced these with the median score of the training data and let the flag capture whether a game was reviewed.

Number of supported languages. Most games support one language, while a few support close to 100, so we applied a log transform to reduce the influence of extreme values.

Platforms and genres. Windows, Mac and Linux support are binary flags. Genres are 14 binary flags, one per genre. A game can belong to several genres at once.

Release month. We one hot encoded the month because months have no meaningful numeric order: December is not "more" than January.

Age. We included the number of years since release to account for time on the market. Older games have had more time to build up owners, and without this feature, the model would confuse newer games with less successful ones. Age replaced release year, since the two are almost perfectly correlated.

Numeric features were standardized, which logistic regression and KNN require, since regularization and distance calculations are sensitive to scale. Random forest is unaffected by scaling. All transformations were fit on the training folds only.

Excluded features. We excluded review counts, recommendations, peak concurrent players and user scores because they are measured after release and directly reflect the outcome. We also excluded Steam Trading Cards, since Steam only enables them once a game passes sales and engagement thresholds. Finally, we excluded price because the dataset records the price at the time the data was collected rather than at launch so it can reflect later or ongoing discounts. Steam is known for having various sales throughout the years, so the pricing snapshot is not providing accurate information.

Methods description

We compared three model families with different underlying mechanisms so that our conclusions would not depend on the assumptions of a single type of model. All three were evaluated against a majority class baseline that always predicts the majority class (Low tier).

Logistic regression (probabilistic, linear). Logistic regression models the probability of each tier as a function of a weighted sum of the features. We chose it as an interpretable baseline. If simple, additive relationships were enough to explain commercial success, it would perform as well as the more flexible models. We tuned the strength of its regularization and compared L2 regularization with L1, which can shrink weak features to zero.

Random forest (tree based ensemble). A random forest trains many decision trees, each on a random sample of games and features and combines their votes to make predictions. We chose it because it can capture nonlinear effects and interactions between features. It also handles a mix of binary and continuous features naturally so it works in our favor.

K nearest neighbors (instance based). KNN does not learn a model in advance. It classifies a game by looking at the most similar games in the training data and makes a decision among them. We chose it because it makes no assumption about the shape of the relationship between features and success. This would be a completely different angle from others approaches, so it may show unexpected results. Because low tier games make up almost 80% of the data, we used balanced class weights in logistic regression and random forest so that errors on the smaller mid and high tiers count more during training. KNN has no equivalent option, which we note when comparing results.

In total, the models use 22 features (Appendix A.1), which become 33 model inputs after release month is one hot encoded into 12 columns. All three model families and the baseline use this same feature set.

Hyperparameter Tuning

We tuned all three model families with grid search and nested cross validation so that model comparison was not biased by the tuning itself. The training set was split into 5 outer folds. Within each outer training fold, a 3 fold grid search chose the hyperparameters with the best macro F1 and the resulting model was then scored on the outer fold it had never seen. This gave five unbiased scores per model family, which we report as mean and standard deviation.

For logistic regression, we searched the regularization strength (C from 0.01 to 10) and L1 versus L2 regularization. For random forest, we searched the maximum tree depth (10, 20 or unlimited), the minimum number of games per leaf (1, 2 or 5) and the share of features considered at each split (square root of the number of features, 30% or 50%). For KNN, we searched the number of neighbors (1, 3, 5 or 15) and whether closer neighbors get more weight in the vote. Our first grid placed the best random forest and KNN settings at its edge, so we widened it and reran the search. The number of trees was fixed at 300 rather than tuned, since random forest performance levels off as more trees are added rather than overfitting.

After selecting the best model family, we tuned it once more with 5 fold cross validation on the full training set and evaluated the resulting model once on the test set.

Metrics

Our main metric is macro F1. We compute an F1 score for each tier, then average the three scores with equal weight. Because 80% of games are in the Low tier, accuracy is misleading. A model that always predicts Low reaches 79.5% accuracy while identifying no Mid or High games. Macro F1 rewards a model only if it performs well on all three tiers. We also report macro precision and macro recall, which show what kind of mistakes a model makes. Low precision means the model labels too many games as Mid or High when they are not. Low recall means the model misses many games that really are Mid or High. From here on, F1, precision and recall refer to these macro averaged values.

Table 1 compares the best tuned model from each family using nested 5 fold cross validation on the training set.

Table 1: Model comparison, nested 5 fold cross validation on the training set

Model	F1	Precision	Recall	Accuracy
Random forest	0.509 ± 0.005	0.495 ± 0.004	0.548 ± 0.007	0.690 ± 0.005
KNN	0.457 ± 0.004	0.493 ± 0.004	0.440 ± 0.004	0.763 ± 0.001
Logistic regression	0.455 ± 0.002	0.446 ± 0.002	0.541 ± 0.004	0.613 ± 0.003
Majority class baseline	0.295 ± 0.000	0.265 ± 0.000	0.333 ± 0.000	0.795 ± 0.000

Values are the mean ± standard deviation across the five outer folds. F1, precision and recall are averaged. Each is computed for every tier, then the three tier scores are averaged with equal weight so the small High tier counts as much as the large Low tier.

As Table 1 shows, all three models clearly outperformed the baseline.

- Logistic regression had high recall but the lowest precision. With class weights it flagged many games as Mid or High, but a straight line relationship between features and tiers could not separate them well.
- KNN had the highest accuracy of the three models but the lowest recall. Without class weights, it tends to predict the large Low tier.
- Random forest had the highest F1, precision and recall.

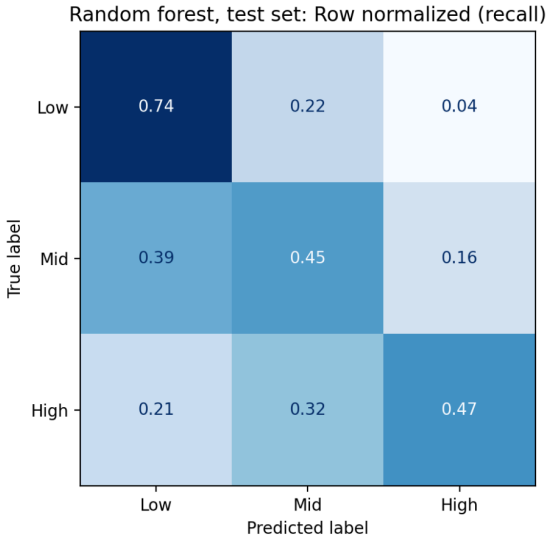


Figure 1: Random forest's confusion matrix on recall

Random forest achieved the highest F1, precision, and recall among the three model families. After tuning on the full training set, the final model used a maximum depth of 20, a minimum of one game per leaf, and the square root of the number of features at each split. On the test set, it achieved an F1 of 0.510 and an accuracy of 0.683, consistent with its cross-validation F1 of 0.509. As shown in Figure 1, recall was 0.74 for Low, 0.45 for Mid, and 0.47 for High.

All subsequent analyses use the final random forest model. Analyses that required retraining the model for comparison (i.e. ablation, sensitivity and the learning curve) used 5 fold cross validation on the training set, with the tuned settings held fixed, so that the test set is not used to compare alternatives. Feature importance and failure analysis used the held out test set, and therefore describe how the model behaves when given unseen data.

Feature Importance and Ablation Analysis

We measured feature importance with permutation importance on the test set. We shuffled one feature at a time, 10 times each, and recorded the average drop in F1. We chose this over random forest's built in impurity importance because it is measured on unseen data. Impurity importance also tends to favor features with many unique values, such as age and language count.

As Figure 2 shows, two features contributed most are age and the number of languages.

- Age: shuffling it lowered F1 by 0.097 ± 0.002, which fits the fact that owner counts build up over time.
- Number of supported languages: shuffling it lowered F1 by 0.052 ± 0.004.

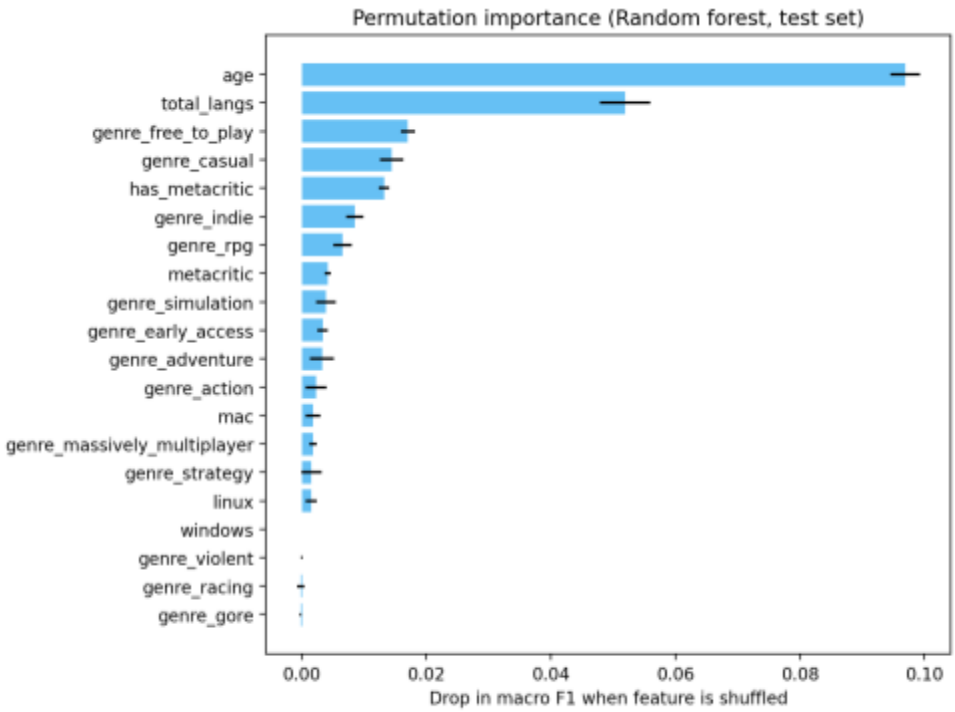


Figure 2: Permutation Importance

The next group was much smaller: the free to play genre (0.017), the casual genre (0.014) and whether a game had a Metacritic score (0.013). The Metacritic score itself contributed little (0.004). Once the model knows whether critics reviewed a game, the score adds little extra information. Release month and several genres, including sports, racing and gore, had importance near zero, within the noise across repeats, so they are not contributing to prediction success. Permutation importance shuffles one column at a time. When features overlap, it can understate them because the model can lean on a related feature instead. We therefore also ran an ablation analysis. We retrained the model with one whole feature group removed at a time.

Table 2: Ablation Results

Feature set	F1	Change
All core features	0.509 ± 0.005	0
Core minus age	0.426 ± 0.003	−0.083
Core minus genres	0.428 ± 0.002	−0.081
Core minus languages	0.464 ± 0.005	−0.045
Core minus Metacritic	0.496 ± 0.006	−0.013
Core minus release month	0.502 ± 0.005	−0.007
Core minus platforms	0.502 ± 0.003	−0.007

Core plus categories	0.536 ± 0.008	0.027
Core plus publisher history	0.526 ± 0.006	0.017
Core plus language flags	0.520 ± 0.006	0.011
Core plus all three groups	0.551 ± 0.004	0.042

As Table 2 shows:

- Genres: they looked minor one flag at a time, but removing all of them lowered F1 by 0.081, almost as much as removing age (0.083).
- Languages: removing them cost 0.045.
- Metacritic: removing the score and flag together cost 0.013.
- Platforms and release month: each cost about 0.007, roughly one standard deviation, so their contribution is weak.

Sensitivity Analysis

We tested how sensitive the model is to its hyperparameters, its features and two other design decisions. For hyperparameters, we changed one setting at a time and kept the rest at their tuned values.

- Maximum depth mattered most. In the grid search, a depth of 10 lowered F1 to about 0.475, and no depth limit lowered it to about 0.497.
- Minimum games per leaf (Figure 3, left): raising it from 1 to 100 lowered F1 gradually, from 0.509 to 0.464, while training F1 fell from 0.80 to 0.47. Larger leaves reduce overfitting, but at a cost in F1.
- Number of trees (Figure 3, right): F1 rose from 0.499 with 25 trees to 0.509 with 400 and was flat after 200, so our fixed choice of 300 trees sits on the plateau.

For features, the ablation in Table 2 shows that results depend most on age, genres and languages. Of the other design decisions, turning off class weights lowered F1 by 0.050, more than any hyperparameter, while dropping the games listed with 0 owners changed F1 by only 0.003, which is within one standard deviation. Overall, the results are robust to most settings and depend mainly on class weights, tree depth and three feature groups.

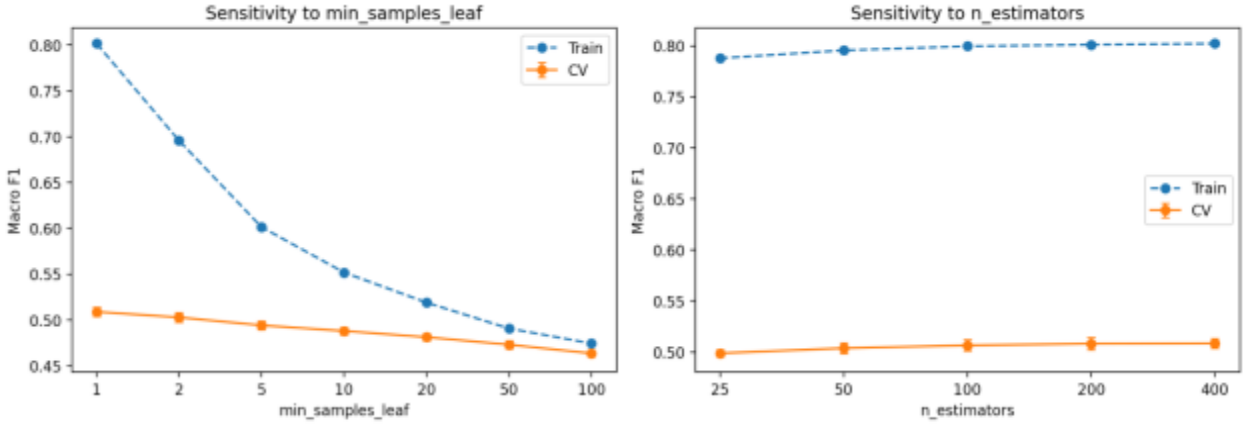


Figure 3: Sensitivity Analysis

Tradeoffs

We identified three tradeoffs.

Balanced performance against accuracy. Class weights make the model pay more attention to Mid and High games, so its accuracy (0.683) is lower than that of the baseline that always predicts Low (0.795). The cost falls on the Low tier. As Figure 1 shows, Low recall is 0.74, and 3,755 Low games in the test set were labeled Mid. We accept this because a model that never finds a successful game is of no use for our question.

Precision against recall for the High tier. As Figure 4 shows, treating High as "High or not", the model's average precision is 0.41, more than six times the 0.064 expected by chance. Precision falls steadily as recall rises:

- To find 20% of High games, the model is right about 65% of the time.
- At the default prediction, it finds 47% of High games with a precision of 0.35.
- To find 80% of High games, precision falls to about 0.15.

The right threshold depends on whether missing a hit or chasing a false lead costs more.

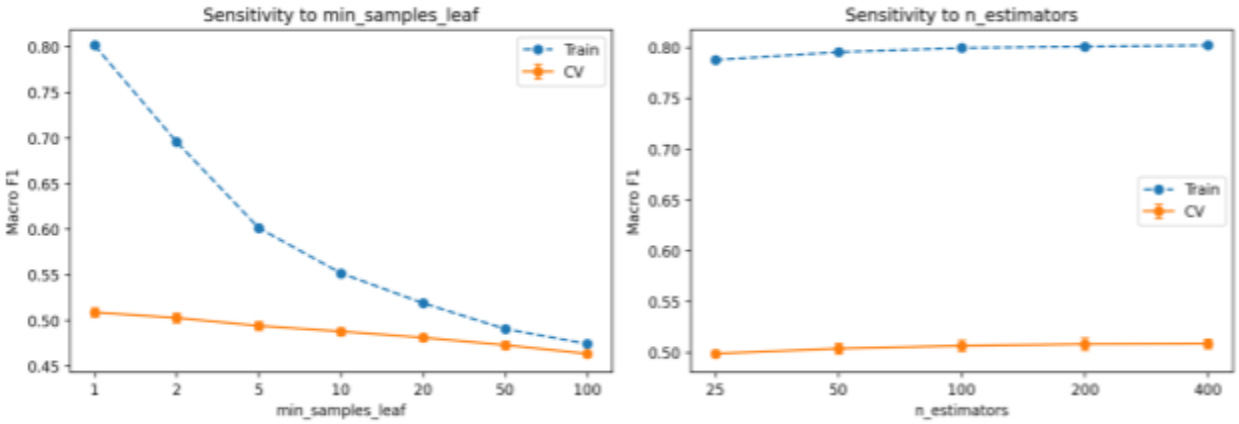


Figure 4: Learning curve of sample count and F1 score (left). Precision vs recall in High tier (right)

Training data size against performance. Figure 4 (left) shows the learning curve. The cross validation F1 (solid line) rises from 0.482 with 3,503 training games to 0.509 with 70,067, and is nearly flat after about 35,000 games. The training F1 (dashed line) falls from 0.96 to 0.80 over the same range, as it becomes harder for the model to fit every training game. The gap between the two lines narrows but remains large, so the model still fits the training games much better than new ones. Because the cross validation line has leveled off, more games of the same kind are unlikely to help much. Table 2 suggests that better features would help more.

Failure Analysis

The final model misclassified 6,952 of 21,897 test games (31.7%). Most mistakes were near misses. As Figure 1 shows, 85% were off by only one tier and the median predicted probability of the wrong tier was about 0.5, so the model was usually uncertain when it was wrong. To find systematic failures, we read the mistakes the model made with the highest confidence and compared the average features of correct and incorrect predictions within each tier. That comparison shows the model's blind spot. High games predicted as Low almost never had a Metacritic score (0.3%, compared with 45% of correctly predicted High games) and they supported fewer languages (5.6 compared with 9.8) and were younger (4.6 years compared with 7.2). Because age, language count and the presence of a Metacritic score carry most of the

signal (Figure 2), games that break this pattern are the ones the model gets wrong with the most confidence. Mistakes fell into four categories:

- Recent breakouts. *Reentry: A Space Flight Simulator* and *Pile Up!* were High tier games released in 2025 with no Metacritic score. Young games are almost always Low, so the model predicted Low with 0.95 probability. A future fix is to add early attention signals not in this dataset, such as wishlist or follower counts.
- Items that are not standalone games. *Mass Effect 2 (2010) Edition* (a rerelease) and *FFXV WINDOWS EDITION 4K Resolution Pack* (an add on) carry the profile of a famous game so the model predicted High, but their owners are split from the original. A future fix is to flag editions and add ons through title keywords or Steam's app type field, and remove them or mark them with a feature.
- Free games. *Slapshot: Rebound* (High, predicted Low) and *Being Mindful* (Mid, predicted High) show that for free games, owners depend more on distribution, such as free claims and giveaways, than on the game's features. A future fix is to model free games separately or add monetization features.
- Tier boundary. *GoNNER* and *Green Hell VR* were well reviewed Mid games just below the 100K cutoff that the model predicted as High. Since owner counts are reported as ranges, games near a cutoff are hard to place. A future fix is an ordinal model that respects the order of the tiers, with one tier off accuracy reported alongside F1.

Because only 2.6% of games have a Metacritic score, the score itself contributed little in the full sample. We wondered how much it tells us among games that critics actually reviewed. We repeated the analysis on the 2,809 reviewed games, using the same split and model setup and compared the random forest with and without the score over 25 cross validation splits. Including the score raised F1 from 0.514 to 0.543 (a gain of 0.029 ± 0.013), and it helped in all 25 splits. The share of High tier games rose from 29% of games scored below 60 to 84% of games scored 90 or above. Among reviewed games, a higher score is associated with more owners. This applies only to that small, unusually successful group, and it describes an association, not an effect of critic scores on sales.

Supervised Learning Discussion

We found that good features and careful framing matter more than model choice. We were surprised that age was the strongest predictor and that the learning curve flattened after about 35,000 games. Genres seemed weak one at a time but mattered almost as much as age as a group. Our main challenges were avoiding leakage and handling class imbalance. We excluded post-release data and snapshot prices, and used macro F1 with balanced class weights since 80% of games were Low tier. With more time, we would add early attention signals like wishlist counts, flag editions and add-ons, and try an ordinal model that respects tier order.

Unsupervised Learning

Workflow Overview

We clustered Steam games on their attributes and used estimated owners only afterward to validate the clusters. Owners was never a clustering input, so any link between clusters and owners can't come from the algorithm having seen the outcome. The steps were:

1. Engineer and preprocess the features
2. Cluster with K-prototypes
3. Test whether K-prototypes cluster membership is associated with estimated owners.
4. Cluster with a latent class model (model-based and probabilistic. It maximizes a likelihood.)
5. Test whether Latent class membership is associated with estimated owners.

Features

Our unsupervised learning model include the following features:

Genres and categories. Multi-hot encoded as binary indicators (e.g., genre_action, cat_multiplayer), since a game can belong to multiple genres and categories at once. total_genres and total_cat add each count as a continuous breadth measure.

Publishers and developers. Not one-hot encoded because thousands of distinct values would create a sparse, high-dimensional feature set. Instead we engineered two numeric proxies for each: prior_games_max (number of other games by that publisher/developer in the dataset, a size proxy) and years_active (years since their first dataset release, an experience proxy), built identically for publishers and developers so both are treated symmetrically. Missing publisher/developer rows default to 0.

Supported languages. Binary indicators for each individually tracked language (English, Chinese, Japanese, Spanish, German, French, Russian, Portuguese, Italian, Korean), plus other_langs for the rest. total_langs adds the total count as a continuous localization measure.

Platforms. Binary indicators for Windows, Mac, and Linux.

The right-skewed numeric features (pub_prior_games_max, publisher_years_active, total_langs, total_categories, dev_prior_games_max, dev_years_active) were log(1+x)-transformed, then all numeric features were standardized to mean 0/std 1. Categorical features stayed unscaled.

K-prototypes (partition-based and non-probabilistic)

Description

K-prototypes combines k-means and k-modes. Each cluster has a prototype made up of a mean for the numeric features and a mode for the categorical features. Each game is assigned to the nearest prototype. Distance is squared Euclidean distance on the numeric features plus γ times the number of categorical mismatches. The algorithm alternates between assigning games and updating prototypes until assignments stop changing.

Why we chose it

Standard k-means assumes all features are continuous, so it treats binary indicators as if they were continuous. Categorical-only methods would force us to bin price and the publisher features, which throws away information. K-prototypes handles both feature types directly. It also scales to a dataset the size of Steam's, and it gives hard, easy-to-read cluster assignments.

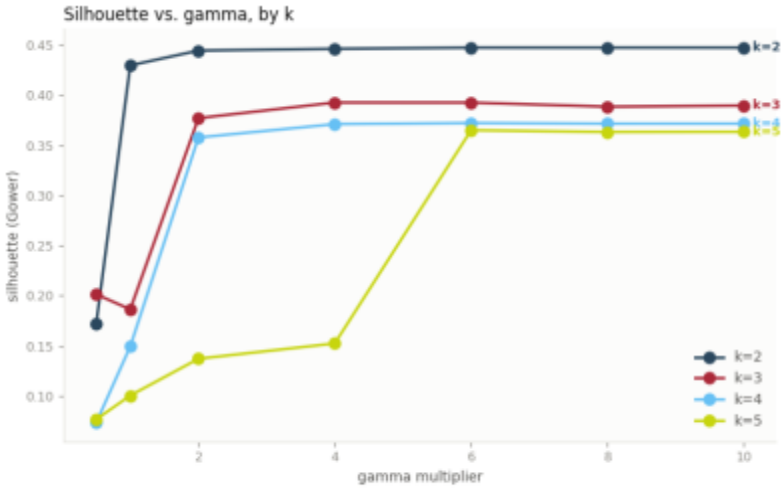


Figure 5: Silhouette vs. gamma, by k

Hyperparameter tuning

- Number of clusters (k): We fit k=2 to 5 and, for each k, computed the average silhouette score using Gower distance, which scales numeric features by their range and compares categorical features by match or mismatch (Euclidean silhouette

isn't meaningful here since our features are mixed).

- Weight on categorical mismatches (γ): We computed an empirical balance point by sampling random pairs of games and setting γ so that the typical numeric distance and the typical categorical mismatch count carry equal weight. We then tested multiples of this balanced value (0.5×, 1×, 2×, 4×, 6×, 8×, 10×) and, for each one, checked two things: whether the silhouette score held up, and whether the resulting cluster assignments stayed stable (measured with the Adjusted Rand Index) as γ changed.
- Initialization: We used Cao initialization with n_init = 10 random restarts and kept the run with the lowest cost, because K-prototypes can converge to a poor local minimum depending on starting centroids.

Model Evaluation: The choice of the metrics

Silhouette score (Gower distance): We selected the optimal number of clusters using the silhouette score. Silhouette score measures how much closer each game is to its own cluster than to the next nearest one (−1 to 1, higher is better). We used Gower distance because our features mix numeric and binary variables.

Adjusted Rand Index (ARI): We checked whether the resulting cluster assignments stayed stable, measured with the Adjusted Rand Index, as γ changed. ARI measures how similar the two modeling results are, from 0 (chance-level agreement) to 1 (identical). It doesn't depend on how clusters are labeled, which matters because "Cluster 1" in one method has no connection to "Class 1" in the other. High agreement suggests the groups are real rather than an artifact of one algorithm.

Model Evaluation: The choice of k and gamma

k = 2 achieved the highest silhouette score at every γ value tested, and was the only k whose cluster assignments converged to a single, stable structure as γ increased. When k= 2, ARI between consecutive γ values approached 1.0 from γ = 6× onward. k = 3, 4, and 5 all showed continued shifts in cluster assignment into the tested γ range, indicating their clustering structure was still sensitive to an essentially arbitrary weighting choice rather than reflecting a robust pattern in the data.

Given these signals, we chose k = 2 and γ = 6× γ_balanced (γ ≈ 8.3). This γ falls within the stable, converged region for k = 2, where silhouette had already plateaued and cluster assignments no longer changed meaningfully with further increases in γ, while without weighting categorical features any more heavily than necessary to reach that stability.

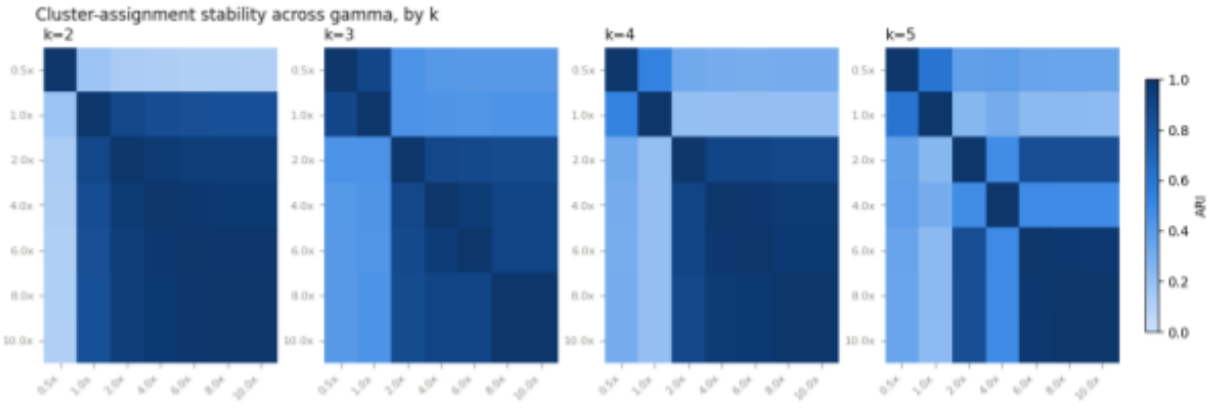


Figure 6: Cluster-assignment stability across gamma, by-k

Model Evaluation: Link to external owners

We regrouped estimated_owners into three ordered groups: Low <20K, Mid 20K–100K, High 100K+, and ran a Kruskal-Wallis test across clusters. We chose this rank-based test because it is appropriate for ordinal data, since owners are reported as ordered ranges rather than exact counts. We reported effect size alongside it since with ~119,000 games almost any difference is statistically significant on its own.

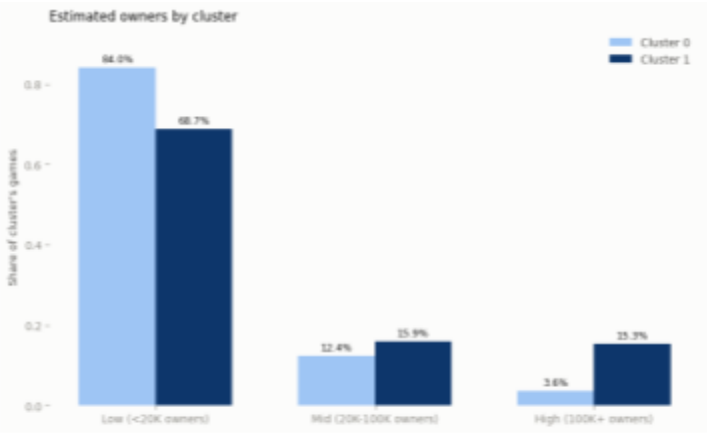


Figure 7: Estimated owners by cluster

As expected, the difference was statistically significant ($H = 3415.72$, $p < .001$) but the effect size was weak ($\varepsilon^2 = 0.029$). This means that cluster membership is associated with the estimated owner group, but only modestly.

Latent class model (probabilistic, model-based)

Description

A latent class model assumes games come from a few hidden "types," each with its own typical profile. For example, 82% of games in one type are multiplayer, while another type is mostly single-player. The model also assumes local independence: within a type, features are unrelated to each other. The model finds these types with the EM algorithm, which repeats two steps until the results stop changing:

- Estimate how likely each game is to belong to each type.
- Update each type's profile based on those estimates.

Each game ends up with a probability for each type (e.g., 90% single-player, 10% multiplayer) and is assigned to the most likely one.

Why we chose it

We chose this approach because it works in a very different way from K-prototypes. K-prototypes minimize a distance, while the latent class model is a generative probability model fit by maximum likelihood. If both methods find similar groups, that is good evidence for us to conclude that the groups exist in the data and are not an artifact of one algorithm. In addition, the latent class model has two other advantages:

- Each class is described by its feature probabilities, so the clusters are easier to interpret.
- Soft assignments show how confidently each game belongs to its class.

Hyperparameter tuning

- Number of classes: We initially fitted LCA models with 2 to 5 classes and compared their Consistent Akaike Information Criterion (CAIC) values (Bozdogan, 1987). CAIC balances model fit and complexity by penalizing additional parameters, helping to avoid adding classes that do not meaningfully improve model fit. Because CAIC continued to decrease through the 5-class model, we expanded our analysis to include up to 10 classes. In total, we evaluated LCA models with 2 to 10 classes.
- Initialization: We ran EM from n_init=10 random starts and kept the solution with the highest log-likelihood, because EM can also get stuck in local optima.

Model Evaluation: The choice of the metrics

Consistent Akaike Information Criterion (CAIC): We selected the number of latent classes using CAIC rather than the standard BIC. CAIC is a similar model-fit criterion (Bozdogan, 1987), but it penalizes additional classes more heavily, which keeps the model from favoring extra classes simply because our large sample (~119,000 games) makes more complex models look statistically better almost by default.

Class size: We additionally required every class in a candidate solution to make up at least 5% of all games. This guards against solutions that technically fit better but do so by splitting off a tiny subgroup rather than identifying a meaningful segment.

Entropy: We used relative entropy to check how confidently each game could be assigned to a single class, scored from 0 to 1, where 1 means every game's class membership is unambiguous. Unlike CAIC and the class-size floor, entropy isn't used to decide the number of classes directly. We use it to check on whether the resulting classes are cleanly separated enough to trust and interpret, rather than classes that overlap heavily with one another in practice.

Model Evaluation: The choice of k

CAIC improved steadily from k = 2 through k = 5, from −10.46M to −14.62M, then a very small change at k = 6 (−14.68M), marking the point where additional classes stopped being justified by the gain in fit. Class sizes stayed above our 5% floor through k = 5, with the smallest class shrinking from 14.8% of games at k = 2 to 5.4% at k = 6. Beyond k = 6, the smallest class fell sharply, down to under 2% by k = 8, indicating the model was producing a very small class. Entropy stayed high throughout (0.977–0.997), so classification certainty was never the limiting factor in this decision. We chose k = 5: the largest number of classes for which CAIC still favored the added complexity, and every resulting class remained large enough to represent a meaningful segment of games.

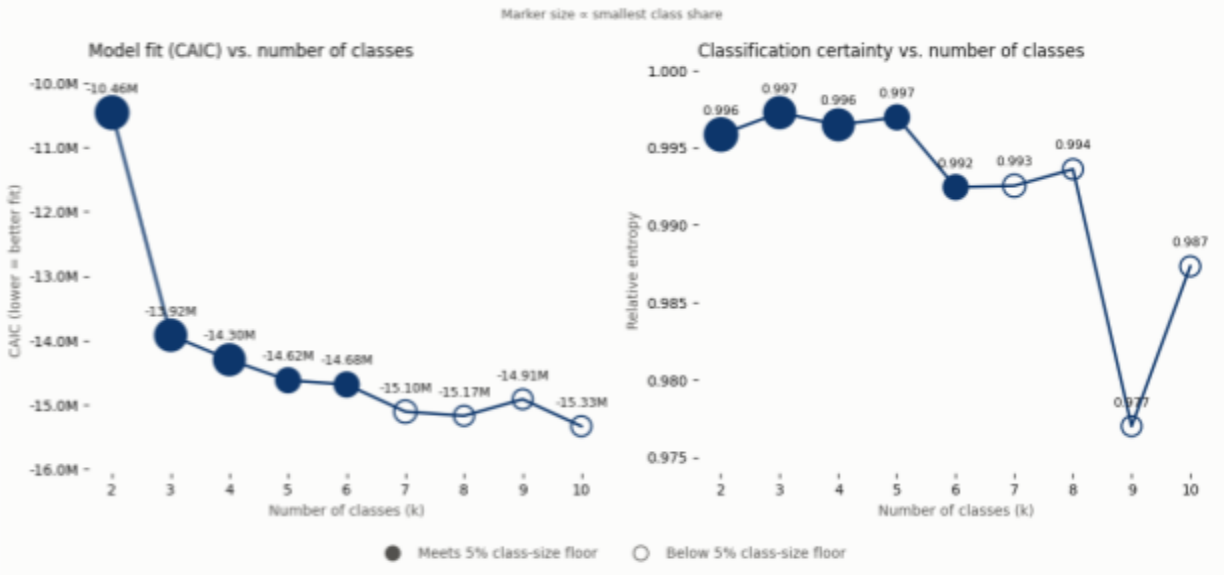


Figure 8: Model fit (left) and classification certainty (right) vs. number of classes

Model Evaluation: Link to estimated owners

Similar to K-prototypes, when we tested whether estimated owners had any association with the LCA classes, we found that the difference was statistically significant ($H = 3975.90$, $p < .001$) but the effect size was weak ($\varepsilon^2 = 0.0334$). This means that class membership is associated with estimated ownership,

but only modestly, which is reasonable given the classes were built purely from catalog features (genres, platforms, languages, publisher/developer history) with no signal tied directly to estimated owners.

Summary of K-prototypes and LCA findings

Although K-prototypes and LCA started from the same features, they landed on different numbers of groups: k = 2 for K-prototypes versus k = 5 for LCA. Despite that disagreement, both methods told a similar story when checked against estimated ownership, a variable excluded from both models. A Kruskal-Wallis test found a statistically significant association between group membership and ownership for both (K-prototypes: $H = 3415.72$; LCA: $H = 3975.90$, ; both $p < .001$), with weak effect sizes in both cases ($\varepsilon^2 = 0.029$ for K-prototypes, $\varepsilon^2 = 0.0334$ for LCA). The fact that two methods that disagree on how many groups exist still independently finding a small-but-real relationship to an unseen outcome is meaningful evidence that both sets of groups reflect genuine structure in the data rather than noise. At the same time, the weak effect sizes in both cases are an honest reminder that catalog features alone (genre, platform, language, publisher/developer history) are a limited predictor of commercial success.

Sensitivity analysis

K-prototypes

To test how sensitive our results were to a hyperparameter choice, we varied γ, the weight K-prototypes places on categorical mismatches relative to numeric distance, across seven multiples of our empirically-derived baseline ($\gamma_{balanced} \approx 1.39$): 0.5×, 1×, 2×, 4×, 6×, 8×, and 10×. For each candidate k (2 through 5), we refit the model at every γ value and measured the Adjusted Rand Index (ARI) between the resulting cluster assignments at each pair of γ values, which tells us how much the clustering itself changed as γ changed, independent of how clusters happened to be labeled.

At low γ multiples (0.5× to 2×), cluster assignments were unstable across every k we tested, including k = 2 (ARI as low as 0.13–0.18 between neighboring γ values), meaning results at this stage were highly sensitive to an essentially arbitrary weighting choice. As γ increased, k = 2 was the only candidate whose assignments converged to a single, stable structure: ARI between consecutive γ values approached 1.0 from $\gamma = 6\times$ onward, meaning the clustering no longer meaningfully changed, even as we continued increasing the categorical weighting. k = 3, 4, and 5, by contrast, kept shifting across the full tested range and never reached that same level of stability. This sensitivity analysis is what ultimately drove our choice of both k and γ: we selected k = 2 specifically because it was the one solution robust to this hyperparameter, and we set $\gamma = 6\times \gamma_{balanced}$ because it is the smallest value within that stable region — avoiding an underweighted-categorical solution that was still sensitive to γ, without pushing γ any higher than necessary to reach that stability.

LCA

To test how sensitive our LCA results were to feature choice, we re-ran the k = 5 model after dropping the 11 language-related indicators and the total_langs numeric summary. We hypothesized these language-related features were contributing to LCA's preference for more classes. We measured the Adjusted Rand Index between the original class assignments and those from the language-dropped model: ARI = 0.549. This indicates moderate stability, with roughly half of the class assignments shifting when language-related features were removed. This suggests that language-related features meaningfully shape, though do not solely determine, the resulting class structure.

Unsupervised Learning Discussion

Given our sample size, we were surprised that K-prototypes optimized at k = 2, and also surprised that catalog features had so little impact on estimated owners (i.e., very weak effect sizes). For K-prototypes, we didn't see any meaningful change in results past k = 5, so we didn't extend the search up to k = 10 the

way we did for LCA. In contrast, LCA kept favoring more classes as k increased, rather than leveling off at k= 2, the way K-prototypes did.
The main challenge for K-prototypes was finding the right γ; we used a grid search to test a range of reasonable values. For LCA, the challenge was that the fit criteria (even with CAIC) kept favoring more classes, with no natural stopping point. We addressed this by adding a minimum class-size requirement as an additional model-selection criterion, so that solutions with classes too small to be substantively meaningful were ruled out regardless of their fit score.
If we had more time, we would increase n_init to 30 to check whether the model results change with more random restarts, and we would increase the sample size used to build the Gower distance matrix (D_gower) for more precise silhouette estimates.

Limitations and Future Work

Although our models identified relationships between game characteristics and estimated ownership, their limited predictive performance suggests that important factors remain unobserved. Future work should prioritize pre-release consumer interest indicators, such as wishlist counts, followers, and social media engagement. NLP techniques could also extract additional information from game descriptions, while historical pricing and monetization data could help explain differences in ownership. Together, these features may improve prediction and provide a more complete understanding of commercial success.

Ethical Considerations

An overarching ethical consideration for our project is how we define success. We used estimated ownership as a proxy for commercial success, but ownership does not necessarily reflect profitability, game quality, or whether a developer achieved their intended goals. For example, a game with 10,000 owners could exceed the expectations of a small independent developer but fall short of those of a larger studio. Similarly, games targeting niche or regional markets may have relatively low ownership while still meeting their developers' financial or creative objectives. Therefore, our findings should be interpreted as measures of commercial reach rather than definitive assessments of a game's overall success or value.

Another concern is the potential for bias in our data and models. Approximately 80% of games fall into the Low success category, creating a risk that our models disproportionately predict lower ownership and overlook games with greater commercial potential. To address this imbalance, we used balanced class weights where applicable and evaluated performance using macro F1, which gives equal importance to each ownership category. Additionally, only approximately 2.6% of games have Metacritic scores, potentially introducing selection bias as games receiving critical attention may already benefit from greater visibility or marketing resources. These biases could disadvantage smaller developers or games targeting niche markets.

Language and geographic representation also introduce ethical concerns. Our LCA sensitivity analysis found that removing language-related features changed cluster assignments (ARI = 0.549), suggesting that language support meaningfully influences how games are grouped. However, language support may reflect a developer's resources, geographic location, or target market rather than a game's quality or commercial potential. If multilingual games are associated with higher ownership, our findings could unintentionally encourage developers to invest in localization or lead investors to favor larger studios, potentially reinforcing existing market inequalities.

Ultimately, our findings are intended to help developers make informed decisions, not prescribe a universal definition of success. Ownership predictions and cluster assignments should be interpreted cautiously, and associations between game characteristics and ownership should not be assumed to represent causal relationships.

Statement of Work

The primary contributor is indicated below for each section of work. Ultimately, this project was a team effort, with all members contributing to discussions, reviewing results, and providing feedback throughout the development of the project and final report.

Introduction: All
EDA and Feature Engineering: Matthew Bellovich
Supervised Learning: Yi-Ju Chen
Unsupervised Learning: Yi-Jui Chen
Ethical Consideration: Matthew Bellovich

References

Al-Hadi, et al. (2026). Video game sales success using random forest based machine learning techniques. *Asian Journal of Research in Computer Science*. https://doi.org/10.9734/ajrcos/2026/v19i4854

Bozdogan, H. (1987). Model selection and Akaike's Information Criterion (AIC): The general theory and its analytical extensions. *Psychometrika* 52, 345–370. https://doi.org/10.1007/BF02294361

Khan, I. (2015). Do Metacritic scores affect game sales? *The Daily Dot*. https://www.dailydot.com/unclick/metacritic-scores-game-sales-gdc-2015/

Ma, J. (2025). Unveiling success drivers in gaming: A machine learning study across Steam, Twitch, and Metacritic. *International Journal of Computer Games Technology*. https://doi.org/10.1155/ijcg/5776632

yvnyli. (2025). *Steam game critic score* [Computer software]. GitHub. https://github.com/yvnyli/steam-game-critic-score

Appendix

Appendix A.1: Features used in supervised learning

Group	Feature (column)	Type	Encoding
Metacritic	has_metacritic	Binary	Unchanged (1 if the game has a Metacritic score)
Metacritic	metacritic	Numeric	Missing scores filled with the training median, then standardized
Languages	total_langs	Numeric	log(1 + x), then standardized
Platforms	windows, mac, linux	Binary (3)	Unchanged
Genres	genre_action, genre_adventure, genre_casual, genre_early_access, genre_free_to_play, genre_indie, genre_massively_multiplayer, genre_racing, genre_rpg, genre_simulation, genre_sports, genre_strategy, genre_violent, genre_gore	Binary (14)	Unchanged
Release timing	release_month	Categorical	One hot encoded (12 columns)
Release timing	age	Numeric	Years between release and data collection, standardized

Appendix A.2: Features used in unsupervised learning

Group	Engineered variables	Type	Transformation & scaling	Rationale
Genres	genre_action, genre_adventure, genre_casual, genre_early_access, genre_free_to_play, genre_indie, genre_massively_multiplayer, genre_racing, genre_rpg, genre_simulation, genre_sports, genre_strategy, genre_violent, genre_gore (14)	Multi-hot binary indicators (0/1)	Unscaled	A game can belong to several genres at once
	total_genres	Continuous count	Standardized only (mean 0, SD 1)	Measures genre breadth
Categories	cat_single_player, cat_multiplayer, cat_coop, cat_online, cat_local_multiplayer, cat_mmo, cat_controller_support, cat_vr, cat_achievements, cat_trading_cards, cat_leaderboards, cat_workshop, cat_in_app_purchases (13)	Multi-hot binary indicators (0/1)	Unscaled	A game can belong to several categories at once
	total_cat	Continuous count	log(1+x), then standardized	Measures category breadth; right-skewed
Publishers	pub_prior_games_max	Numeric proxy: other games by the publisher in the dataset	log(1+x), then standardized	Size proxy; avoids sparse one-hot encoding of thousands of publishers

Group	Engineered variables	Type	Transformation & scaling	Rationale
	publisher_years_active	Numeric proxy: years since the publisher's first dataset release	log(1+x), then standardized	Experience proxy
Developers	dev_prior_games_max	Numeric proxy: other games by the developer in the dataset	log(1+x), then standardized	Size proxy; built the same way as for publishers
	dev_years_active	Numeric proxy: years since the developer's first dataset release	log(1+x), then standardized	Experience proxy
Supported languages	lang_english, lang_chinese, lang_japanese, lang_spanish, lang_german, lang_french, lang_russian, lang_portuguese, lang_italian, lang_korean (10)	Binary indicators (0/1)	Unscaled	Tracks the most common localization targets individually
	other_langs	Binary indicator (0/1)	Unscaled	Captures any language not tracked individually
	total_langs	Continuous count	log(1+x), then standardized	Measures localization breadth; right-skewed

Group	Engineered variables	Type	Transformation & scaling	Rationale
Platforms	windows, mac, linux (3)	Binary indicators (0/1)	Unscaled	Platform availability